Specification: Identification of service interfaces during the migration from monoliths to microservices

Alexander Ley
Computer Science Department
University of applied science Bonn-Rhein-Sieg
Sankt Augustin, Germany
s6alleyy@outlook.de

I. Context and Motivation

In Software Engineering of business applications resulting software products are traditionally built as one single deployable unit. This kind of software organization is called a *Monolithical Architecture*. [1], [2]

Enterprise Applications are often made of the three following parts [1]:

1) User Interface (UI)
2) Business Logic Layer (BL)
3) Database (DB)

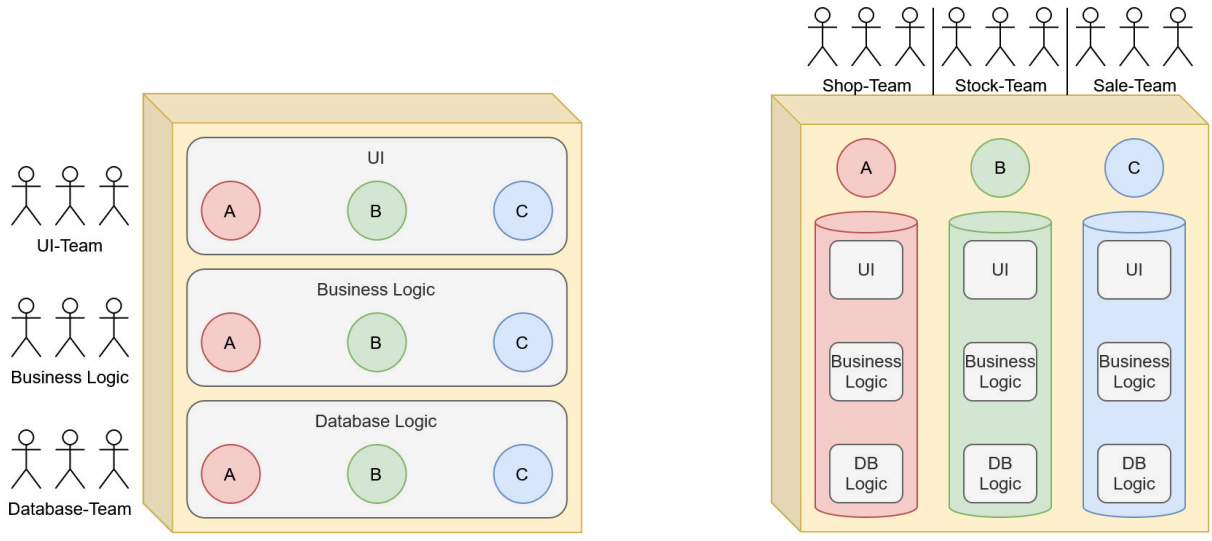
As software is traditionally organized and split around these technical layers the team maintaining the software is split likewise according these layers leading to "UI Teams", "BL Teams" and "DB Teams". Figure Fig. 1 provides on the left an illustrations of a Monolithical Architecture. [1]

Whenever the software has to be adapted all teams might be involved even when the changes might be small. According to Conway's Law which is quoted in Quote 1 the communication structure of software team will become mirrored in the software product itself.

> "Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure."

Quote 1. Conway's Law quoted by Melvin Conway, 1968 [1]

As software systems grow large and becomes more and more complex the above presented Monolithical Architectures is resulting according Conway's Law into a software product which is *highly coupled* and *less cohesive*. *Highly coupled* means that the software product has many dependecies to other parts of the software and *less cohesive* means that one component not have a single responsibility and might be responsable for several things. In order words the software logic is scattered all over the system. [1], [2]



(a) Monolithical Architecture

(b) Microservice Architecture

Fig. 1. Difference between a *Monolithical Architecture (left)* and a *Microservice Architecture (right)*. Orifinal illustration by C. Nockemann [3].

Therefore when a monolith grow large desired features like *scalability*, *maintenance* and *deployability* suffers.

The *Microservice Architecture* on the other hand focus to address the above mentioned downsides of monoliths. As illustrated in figure Fig. 1 on the right, Microservices favor to decompose systems into several, relatively small components which are organized around business capabilities. These components can be maintained, scaled and deployed individually. [1], [2]

Therefore, companies who have invested in a monolithical architecture might consider to migrate and decompose their software into microservices [2]. However, migrating to microservices is a resource intensive task involving identification of service boundaries within the monolith and package found candidates into self-contained microservices with a defined API [4].

NOTES:

A. Migration

- companies having invested in a monolithtic architectures might consider to migrate to microservices [2]
- however migrating to microservices is a resource intensive task involving the identification of service boundaries and packaging them into self-contained microservices with defined API's [4]

II. Probleme / Fragestellungen

Was genau ist Ihre Problemstellung? Welche Fragestelllungen behandeln Sie in Ihrer Arbeit?

NOTES:

- The authors I. Trabelsi, B. Mahmoudi, J. B. Minani, N. Moha, and Y.-G. Guéhéneuc [4] acknowledge that some phases such as Service Identification which is also called service decomposition are extensively studied while some other phases like Packaging Microservices and generating necessary code for microservice API is less well studied [4]
- a first ai guided literature review shows that so far no existing SLR exists which focus on service interfaces / API after the service identification migration phase. Some literature might only shortly mention some desired aspects like API's without the migration background or dealing with Packaging Phase and deploying without dedicated focus on service interfaces
- the authors I. Trabelsi, B. Mahmoudi, J. B. Minani, N. Moha, and Y.-G. Guéhéneuc [4] focus on Machine Learning approaches during the migration phase
- the authors Y. Abgaz *et al.* [2] seek to give a general overview about the field of Monolith Decomposition without a special focus on interfaces
- the authors M. Abdellatif *et al.* [5] focus to create a taxonomy of different Service-Identification methods without a focus on the interfaces
- an SLR with dedicated focus on service interfaces after the service identification seems in the migration to be missing
- this work is seeking to enlight that gap and seek to present how far research got in regard to interface design after the service decomposition step as it is important phase in the migration process
- focus on the following questions:
  - Which methods and procedures does exist to define and implement service interfaces after the Service-Identifikation phase in the migration from Monoliths to Microservices?
    - How mature are there?
  - Which types, patterns amd metrics are used in the examined methods and procedures?
  - How does the methods and procedures are evaluated?

III. Methodisches Vorgehen zur Zielerreichung

Wie gehen Sie bei der Erstellung der Ergebnisse vor (Arbeitsschritte) und welche Methodik praktizieren Sie dabei?

NOTES:

IV. Ziele und angestrebte Ergebnisse

Welche Ziele verfolgen Sie mit der Beantwortung der behandelten Fragen und welche wesentlichen Ergebnisse entstehen dabei?

NOTES:

- Developing a more detailed view over dedicated SLR's

V. Wissenschaftlicher Beitrag

Welchen Beitrag zur Bewältigung der Problemstellung leistet Ihre Arbeit? Welche neuen Erkenntnisse können Lesende Ihrer Arbeit erwarten?

References

[1] J. Lewis and M. Fowler, "Microservices: A Definition of This New Architectural Term." Accessed: Aug. 25, 2026. [Online]. Available: https://martinfowler.com/articles/microservices.html

[2] Y. Abgaz *et al.*, "Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4213–4242, Aug. 2023, doi: 10.1109/TSE.2023.3287297.

[3] C. Nockemann, "Domain-Driven Design in Der Praxis," in *Presentations during the Lecture of Object-Oriented Component Architectures (OOKA) Renamed to Service-Oriented Component Architectures (SEKA)*, University of applied science Bonn-Rhein-Sieg, 2025.

[4] I. Trabelsi, B. Mahmoudi, J. B. Minani, N. Moha, and Y.-G. Guéhéneuc, "A Systematic Literature Review of Machine Learning Approaches for Migrating Monolithic Systems to Microservices," *IEEE Transactions on Software Engineering*, vol. 51, no. 11, pp. 2972–2995, Nov. 2025, doi: 10.1109/TSE.2025.3603897.

[5] M. Abdellatif *et al.*, "A Taxonomy of Service Identification Approaches for Legacy Software Systems Modernization," *Journal of Systems and Software*, vol. 173, p. 110868, Mar. 2021, doi: 10.1016/j.jss.2020.110868.